

Executing shell commands from Node.js

[2022-07-05] [dev](#), [javascript](#), [nodejs](#)

(Ad, please don't block)

 Adobe Stock



Get 10 Free Images From Adobe Stock. Start Now.

ADS VIA CARBON

Warning: This blog post is outdated. Instead, read [chapter “Running shell commands in child processes”](#) in “Shell scripting with Node.js”.

In this blog post, we'll explore how we can execute shell commands from Node.js, via module `'node:child_process'`.

-
- 1 Overview of this blog post
 - 1.1 Windows vs. Unix
 - 1.2 Functionality we often use in the examples
 - 2 Spawning processes asynchronously: `spawn()`
 - 2.1 How `spawn()` works
 - 2.2 When is the shell command executed?
 - 2.3 Command-only mode vs. `args` mode
 - 2.4 Sending data to the `stdin` of the child process
 - 2.5 Piping manually
 - 2.6 Handling unsuccessful exits (including errors)
 - 2.7 Waiting for the exit of a child process
 - 2.8 Terminating child processes
 - 3 Spawning processes synchronously: `spawnSync()`
 - 3.1 When is the shell command executed?
 - 3.2 Reading from `stdout`
 - 3.3 Sending data to the `stdin` of the child process
 - 3.4 Handling unsuccessful exits (including errors)
 - 4 Asynchronous helper functions based on `spawn()`
 - 4.1 `exec()`
 - 4.2 `execFile()`
 - 5 Synchronous helper functions based on `spawnSync()`

5.1 `execSync()`

5.2 `execFileSync()`

6 Useful libraries

6.1 `tinysh`: a helper for spawning shell commands

6.2 `node-powershell`: executing Windows PowerShell commands via Node.js

7 How to choose between the functions of module `'node:child_process'`

1 Overview of this blog post

Module `'node:child_process'` has a function for executing shell commands (in *spawned* child processes) that comes in two versions:

- An asynchronous version `spawn()`.
- A synchronous version `spawnSync()`.

We'll first explore `spawn()` and then `spawnSync()`. We'll conclude by looking at the following functions that are based on them and relatively similar:

- Based on `spawn()`:
 - `exec()`
 - `execFile()`
- Based on `spawnSync()`:
 - `execSync()`
 - `execFileSync()`

1.1 Windows vs. Unix

The code shown in this blog post runs on Unix, but I have also tested it on Windows – where most of it works with minor changes (such as ending lines with `'\r\n'` instead of `'\n'`).

1.2 Functionality we often use in the examples

The following functionality shows up often in the examples. That's why it's explained here, once:

- Assertions: `assert.equal()` for primitive values and `assert.deepEqual()` for objects. The necessary import is never shown in the examples:

```
import * as assert from 'node:assert/strict';
```

- Function `Readable.toWeb()` converts Node's native `stream.Readable` to a web stream (an instance of `ReadableStream`). It is explained in [the blog post on web streams](#). `Readable` is always imported in the examples.
- The asynchronous function `readableStreamToString()` consumes a readable web stream and returns a string (wrapped in a Promise). It is explained in [the blog post on web streams](#). This function is assumed to be available in the examples.

2 Spawning processes asynchronously: `spawn()`

2.1 How `spawn()` works

```
spawn(  
  command: string,  
  args?: Array<string>,  
  options?: Object  
) : ChildProcess
```

`spawn()` asynchronously executes a command in a new process: The process runs concurrently to Node's main JavaScript process and we can communicate with it in various ways (often via streams).

Next, there is documentation for the parameters and the result of `spawn()`. If you prefer to learn by example, you can skip that content and continue with the subsections that follow.

2.1.1 Parameter: `command`

`command` is a string with the shell command. There are two modes of using this parameter:

- Command-only mode: `args` is omitted and `command` contains the whole shell command. We can even use shell features such as piping between multiple executables, redirecting I/O into files, variables, and wildcards.
 - `options.shell` must be `true` because we need a shell to handle the shell features.
- Args mode: `command` contains only the name of the command and `args` contains its arguments.

- If `options.shell` is `true`, many meta-characters inside arguments are interpreted and features such as wildcards and variable names work.
- If `options.shell` is `false`, strings are used verbatim and we never have to escape meta-characters.

Both modes are demonstrated later in this post.

2.1.2 Parameter: `options`

The following `options` are most interesting:

- `.shell: boolean|string (default: false)`
Should a shell be used to execute the command?
 - On Windows, this option should almost always be `true`. For example, `.bat` and `.cmd` files cannot be executed otherwise.
 - On Unix, only core shell features (e.g. piping, I/O redirection, filename wildcards, and variables) are not available if `.shell` is `false`.
 - If `.shell` is `true`, we have to be careful with user input and sanitize it because it's easy to execute arbitrary code. We also have to escape meta-characters if we want to use them as non-meta-characters.
 - We can also set `.shell` to the path of a shell executable. Then Node.js uses that executable to execute the command. If we set `.shell` to `true`, Node.js uses:
 - Unix: `process.env.SHELL`
 - Windows: `process.env.ComSpec`
- `.cwd: string | URL`
Specifies the *current working directory* (CWD) to use while executing the command.
- `.stdio: Array<string|Stream>|string`
Configures how standard I/O is set up. This is explained below.
- `.env: Object (default: process.env)`
Lets us specify shell variables for the child process. Tips:
 - Look at `process.env` (e.g. in the Node.js REPL) to see what variables exist.
 - We can use spreading to non-destructively override an existing variable – or create it if it doesn't exist yet:

```
{env: {...process.env, MY_VAR: 'Hi!'}}
```

- `.signal: AbortSignal`
If we create an `AbortController` `ac`, we can pass `ac.signal` to `spawn()` and

abort the child process via `ac.abort()`. That is demonstrated later in this post.

- `.timeout: number`

If the child process takes longer than `.timeout` milliseconds, it is killed.

2.1.3 `options.stdio`

Each of the standard I/O streams of the child process has a numeric ID, a so-called *file descriptor*:

- Standard input (stdin) has the file descriptor 0.
- Standard output (stdout) has the file descriptor 1.
- Standard error (stderr) has the file descriptor 2.

There can be more file descriptors, but that's rare.

`options.stdio` configures if and how the streams of the child process are piped to streams in the parent process. It can be an Array where each element configures the file descriptor that is equal to its index. The following values can be used as Array elements:

- `'pipe'`:
 - Index 0: Pipe `childProcess.stdin` to the child's stdin. Note that, despite its name, the former is a stream that belongs to the parent process.
 - Index 1: Pipe the child's stdout to `childProcess.stdout`.
 - Index 2: Pipe the child's stderr to `childProcess.stderr`.
- `'ignore'`: Ignore the child's stream.
- `'inherit'`: Pipe the child's stream to the corresponding stream of the parent process.
 - For example, if we want the child's stderr to be logged to the console, we can use `'inherit'` at index 2.
- Native Node.js stream: Pipe to or from that stream.
- Other values are supported, too, but that's beyond the scope of this post.

Instead of specifying `options.stdio` via an Array, we can also abbreviate:

- 'pipe' is equivalent to ['pipe', 'pipe', 'pipe'] (the default for `options.stdio`).
- 'ignore' is equivalent to ['ignore', 'ignore', 'ignore'].
- 'inherit' is equivalent to ['inherit', 'inherit', 'inherit'].

2.1.4 Result: instance of `ChildProcess`

`spawn()` returns instances of `ChildProcess`.

Interesting data properties:

- `.exitCode: number | null`
Contains the code with which the child process exited:
 - 0 (zero) means normal exit.
 - A number greater than zero means an error happened.
 - `null` means the process hasn't exited yet.
- `.signalCode: string | null`
The POSIX signal with which a child process was killed or `null` if it wasn't. See the description of method `.kill()` below for more information.
- Streams: Depending on how standard I/O is configured (see previous subsection), the following streams become available:
 - `.stdin`
 - `.stdout`
 - `.stderr`
- `.pid: number | undefined`
The *process identifier* (PID) of the child process. If spawning fails, `.pid` is `undefined`. This value is available immediately after calling `spawn()`.

Interesting methods:

- `.kill(signalCode?: number | string = 'SIGTERM'): boolean`
Sends a POSIX signal to the child process (which usually results in the termination of the process):
 - The man page for `signal` contains a list of values.
 - Windows does not support signals, but Node.js emulates some of them – e.g.: `SIGINT`, `SIGTERM`, and `SIGKILL`. For more information, see the Node.js documentation.

This method is demonstrated later in this post.

Interesting events:

- `.on('exit', (exitCode: number|null, signalCode: string|null) => {})`

This event is emitted after the child process ends:

- The callback parameters provide us with either the exit code or the signal code: One of them will always be non-null.
- Some of its standard I/O streams might still be open because multiple processes might share the same streams. Event `'close'` notifies us when all stdio streams are closed after the exit of a child process.

- `.on('error', (err: Error) => {})`

This event is most commonly emitted if a process could not be spawned (see [example](#) later) or the child process could not be killed. An `'exit'` event may or may not be emitted after this event.

We'll see later how events can be turned into Promises that can be awaited.

2.2 When is the shell command executed?

When using the asynchronous `spawn()`, the child process for the command is started asynchronously. The following code demonstrates that:

```
import {spawn} from 'node:child_process';

spawn(
  'echo', ['Command starts'],
  {
    stdio: 'inherit',
    shell: true,
  }
);
console.log('After spawn()');
```

This is the output:

```
After spawn()
Command starts
```

2.3 Command-only mode vs. args mode

In this section, we specify the same command invocation in two ways:

- **Command-only mode:** We provide the whole invocation via the first parameter `command`.

- **Args mode:** We provide the command via the first parameter `command` and its arguments via the second parameter `args`.

2.3.1 Command-only mode

```
import {Readable} from 'node:stream';
import {spawn} from 'node:child_process';

const childProcess = spawn(
  'echo "Hello, how are you?"',
  {
    shell: true, // (A)
    stdio: ['ignore', 'pipe', 'inherit'], // (B)
  }
);

const stdout = Readable.toWeb(
  childProcess.stdout.setEncoding('utf-8'));

// Result on Unix
assert.equal(
  await readableStreamToString(stdout),
  'Hello, how are you?\n' // (C)
);

// Result on Windows: '"Hello, how are you?"\r\n'
```

Each command-only spawning with arguments requires `.shell` to be `true` (line A) – even if it's as simple as this one.

In line B, we tell `spawn()` how to handle standard I/O:

- Ignore standard input.
- Pipe the child process `stdout` to `childProcess.stdout` (a stream that belongs to the parent process).
- Pipe child process `stderr` to parent process `stderr`.

In this case, we are only interested in the output of the child process. Therefore, we are done once we have processed the output. In other cases, we might have to wait until the child exits. How to do that, is demonstrated later.

In command-only mode, we see more peculiarities of shells – for example, the Windows Command shell output includes double quotes (last line).

2.3.2 Args mode

```
import {Readable} from 'node:stream';
import {spawn} from 'node:child_process';

const childProcess = spawn(
  'echo', ['Hello, how are you?'],
  {
    shell: true,
    stdio: ['ignore', 'pipe', 'inherit'],
  }
);
const stdout = Readable.toWeb(
  childProcess.stdout.setEncoding('utf-8'));

// Result on Unix
assert.equal(
  await readableStreamToString(stdout),
  'Hello, how are you?\n'
);
// Result on Windows: 'Hello, how are you?\r\n'
```

2.3.3 Meta-characters in args

Let's explore what happens if there are meta-characters in args:

```
import {Readable} from 'node:stream';
import {spawn} from 'node:child_process';

async function echoUser({shell, args}) {
  const childProcess = spawn(
    `echo`, args,
    {
      stdio: ['ignore', 'pipe', 'inherit'],
      shell,
    }
  );
  const stdout = Readable.toWeb(
    childProcess.stdout.setEncoding('utf-8'));
  return readableStreamToString(stdout);
}
```

```

    }

    // Results on Unix
    assert.equal(
      await echoUser({shell: false, args: ['$USER']}), // (A)
      '$USER\n'
    );
    assert.equal(
      await echoUser({shell: true, args: ['$USER']}), // (B)
      'rauschma\n'
    );
    assert.equal(
      await echoUser({shell: true, args: [String.raw`\$USER`]}), // (C)
      '$USER\n'
    );

```

- If we don't use a shell, meta-characters such as the dollar sign (\$) have no effect (line A).
- With a shell, \$USER is interpreted as a variable (line B).
- If we don't want that, we have to escape the dollar sign via a backslash (line C).

Similar effects occur with other meta-characters such as asterisks (*).

These were two examples of Unix shell meta-characters. Windows shells have their own meta-characters and their own ways of escaping.

2.3.4 A more complicated shell command

Let's use more shell features (which requires command-only mode):

```

import {Readable} from 'node:stream';
import {spawn} from 'node:child_process';
import {EOL} from 'node:os';

const childProcess = spawn(
  `(echo cherry && echo apple && echo banana) | sort`,
  {
    stdio: ['ignore', 'pipe', 'inherit'],
    shell: true,
  }
)

```

```
);  
const stdout = Readable.toWeb(  
  childProcess.stdout.setEncoding('utf-8'));  
assert.equal(  
  await readableStreamToString(stdout),  
  'apple\nbanana\ncherry\n'  
);
```

2.4 Sending data to the stdin of the child process

So far, we have only read the standard output of a child process. But we can also send data to standard input:

```
import {Readable, Writable} from 'node:stream';  
import {spawn} from 'node:child_process';  
  
const childProcess = spawn(  
  `sort`, // (A)  
  {  
    stdio: ['pipe', 'pipe', 'inherit'],  
  }  
);  
const stdin = Writable.toWeb(childProcess.stdin); // (B)  
const writer = stdin.getWriter(); // (C)  
try {  
  await writer.write('Cherry\n');  
  await writer.write('Apple\n');  
  await writer.write('Banana\n');  
} finally {  
  writer.close();  
}  
  
const stdout = Readable.toWeb(  
  childProcess.stdout.setEncoding('utf-8'));  
assert.equal(  
  await readableStreamToString(stdout),  
  'Apple\nBanana\nCherry\n'  
);
```

We use the shell command `sort` (line A) to sort lines of text for us.

In line B, we use `Writable.toWeb()` to convert a native Node.js stream to a web stream (see [the blog post on web streams for more information](#)).

How to write to a `WritableStream` via a writer (line C) is also explained in [the blog post on web streams](#).

2.5 Piping manually

We previously let a shell execute the following command:

```
(echo cherry && echo apple && echo banana) | sort
```

In the following example, we do the piping manually, from the echoes (line A) to the sorting (line B):

```
import {Readable, Writable} from 'node:stream';
import {spawn} from 'node:child_process';

const echo = spawn( // (A)
  `echo cherry && echo apple && echo banana`,
  {
    stdio: ['ignore', 'pipe', 'inherit'],
    shell: true,
  }
);

const sort = spawn( // (B)
  `sort`,
  {
    stdio: ['pipe', 'pipe', 'inherit'],
    shell: true,
  }
);

//==== Transferring chunks from echo.stdout to sort.stdin ====

const echoOut = Readable.toWeb(
  echo.stdout.setEncoding('utf-8'));
const sortIn = Writable.toWeb(sort.stdin);

const sortInWriter = sortIn.getWriter();
try {
```

```

    for await (const chunk of echoOut) { // (C)
        await sortInWriter.write(chunk);
    }
} finally {
    sortInWriter.close();
}

//==== Reading sort.stdout ====

const sortOut = Readable.toWeb(
    sort.stdout.setEncoding('utf-8'));
assert.equal(
    await readableStreamToString(sortOut),
    'apple\nbanana\ncherry\n'
);

```

ReadableStreams such as `echoOut` are asynchronously iterable. That's why we can use a `for-await-of` loop to read their *chunks* (the fragments of the streamed data). For more information, see [the blog post on web streams](#).

2.6 Handling unsuccessful exits (including errors)

There are three main kinds of unsuccessful exits:

- The child process can't be spawned.
- An error happens in the shell.
- A process is killed.

2.6.1 The child process can't be spawned

The following code demonstrates what happens if a child process can't be spawned. In this case, the cause is that the shell's path doesn't point to an executable (line A).

```

import {spawn} from 'node:child_process';

const childProcess = spawn(
    'echo hello',
    {
        stdio: ['inherit', 'inherit', 'pipe'],
        shell: '/bin/does-not-exist', // (A)
    }
);

```

```

    }
  );
  childProcess.on('error', (err) => { // (B)
    assert.equal(
      err.toString(),
      'Error: spawn /bin/does-not-exist ENOENT'
    );
  });
});

```

This is the first time that we use events to work with child processes. In line B, we register an event listener for the 'error' event. The child process starts after the current code fragment is finished. That helps prevent race conditions: When we start listening we can be sure that the event hasn't been emitted yet.

2.6.2 An error happens in the shell

If the shell code contains an error, we don't get an 'error' event (line B), we get an 'exit' event with a non-zero exit code (line A):

```

import {Readable} from 'node:stream';
import {spawn} from 'node:child_process';

const childProcess = spawn(
  'does-not-exist',
  {
    stdio: ['inherit', 'inherit', 'pipe'],
    shell: true,
  }
);
childProcess.on('exit',
  async (exitCode, signalCode) => { // (A)
    assert.equal(exitCode, 127);
    assert.equal(signalCode, null);
    const stderr = Readable.toWeb(
      childProcess.stderr.setEncoding('utf-8'));
    assert.equal(
      await readableStreamToString(stderr),
      '/bin/sh: does-not-exist: command not found\n'
    );
  }
);

```

```
childProcess.on('error', (err) => { // (B)
  console.error('We never get here!');
});
```

2.6.3 A process is killed

If a process is killed on Unix, the exit code is `null` (line C) and the signal code is a string (line D):

```
import {Readable} from 'node:stream';
import {spawn} from 'node:child_process';

const childProcess = spawn(
  'kill $$', // (A)
  {
    stdio: ['inherit', 'inherit', 'pipe'],
    shell: true,
  }
);

console.log(childProcess.pid); // (B)
childProcess.on('exit', async (exitCode, signalCode) => {
  assert.equal(exitCode, null); // (C)
  assert.equal(signalCode, 'SIGTERM'); // (D)
  const stderr = Readable.toWeb(
    childProcess.stderr.setEncoding('utf-8'));
  assert.equal(
    await readableStreamToString(stderr),
    '' // (E)
  );
});
```

Note that there is no error output (line E).

Instead of the child process killing itself (line A), we could have also paused it for a longer time and killed it manually via the process ID that we logged in line B.

What happens if we kill a child process on Windows?

- `exitCode` is 1.
- `signalCode` is `null`.

2.7 Waiting for the exit of a child process

Sometimes we only want to wait until a command is finished. That can be achieved via events and via Promises.

2.7.1 Waiting via events

```
import * as fs from 'node:fs';
import {spawn} from 'node:child_process';

const childProcess = spawn(
  `(echo first && echo second) > tmp-file.txt`,
  {
    shell: true,
    stdio: 'inherit',
  }
);

childProcess.on('exit', (exitCode, signalCode) => { // (A)
  assert.equal(exitCode, 0);
  assert.equal(signalCode, null);
  assert.equal(
    fs.readFileSync('tmp-file.txt', {encoding: 'utf-8'}),
    'first\nsecond\n'
  );
});
```

We are using the standard Node.js event pattern and register a listener for the 'exit' event (line A).

2.7.2 Waiting via Promises

```
import * as fs from 'node:fs';
import {spawn} from 'node:child_process';

const childProcess = spawn(
  `(echo first && echo second) > tmp-file.txt`,
  {
    shell: true,
    stdio: 'inherit',
  }
);

const {exitCode, signalCode} = await onExit(childProcess); // (A)
```



```

assert.equal(exitCode, 0);
assert.equal(signalCode, null);
assert.equal(
  fs.readFileSync('tmp-file.txt', {encoding: 'utf-8'}),
  'first\nsecond\n'
);

```

The helper function `onExit()` that we use in line A, returns a Promise that is fulfilled if an `'exit'` event is emitted:

```

export function onExit(eventEmitter) {
  return new Promise((resolve, reject) => {
    eventEmitter.once('exit', (exitCode, signalCode) => {
      if (exitCode === 0) { // (B)
        resolve({exitCode, signalCode});
      } else {
        reject(new Error(
          `Non-zero exit: code ${exitCode}, signal ${signalCode}`));
      }
    });
    eventEmitter.once('error', (err) => { // (C)
      reject(err);
    });
  });
}

```

If `eventEmitter` fails, the returned Promise is rejected and `await` throws an exception in line A. `onExit()` handles two kinds of failures:

- `exitCode` isn't zero (line B). That happens:
 - If there is a shell error. Then `exitCode` is greater than zero.
 - If the child process is killed on Unix. Then `exitCode` is null and `signalCode` is non-null.
 - Killing child process on Windows produces a shell error.
- An `'error'` event is emitted (line C). That happens if the child process can't be spawned.

2.8 Terminating child processes

2.8.1 Terminating a child process via an AbortController

In this example, we use an AbortController to terminate a shell command:

```
import {spawn} from 'node:child_process';

const abortController = new AbortController(); // (A)

const childProcess = spawn(
  `echo Hello`,
  {
    stdio: 'inherit',
    shell: true,
    signal: abortController.signal, // (B)
  }
);
childProcess.on('error', (err) => {
  assert.equal(
    err.toString(),
    'AbortError: The operation was aborted'
  );
});
abortController.abort(); // (C)
```

We create an AbortController (line A), pass its signal to `spawn()` (line B), and terminate the shell command via the AbortController (line C).

The child process starts asynchronously (after the current code fragment is executed). That's why we can abort before the process has even started and why we don't see any output in this case.

2.8.2 Terminating a child process via `.kill()`

In the next example, we terminate a child process via the method `.kill()` (last line):

```
import {spawn} from 'node:child_process';

const childProcess = spawn(
  `echo Hello`,
  {

```

```

    stdio: 'inherit',
    shell: true,
  }
);
childProcess.on('exit', (exitCode, signalCode) => {
  assert.equal(exitCode, null);
  assert.equal(signalCode, 'SIGTERM');
});
childProcess.kill(); // default argument value: 'SIGTERM'

```

Once again, we kill the child process before it has started (asynchronously!) and there is no output.

3 Spawning processes synchronously:

`spawnSync()`

```

spawnSync(
  command: string,
  args?: Array<string>,
  options?: Object
): Object

```

`spawnSync()` is the synchronous version of `spawn()` – it waits until the child process exits before it synchronously(!) returns an object.

The parameters are mostly the same as those of `spawn()`. `options` has a few additional properties – e.g.:

- `.input: string | TypedArray | DataView`
If this property exists, its value is sent to the standard input of the child process.
- `.encoding: string (default: 'buffer')`
Specifies the encoding that is used for all standard I/O streams.

The function returns an object. Its most interesting properties are:

- `.stdout: Buffer | string`
Contains whatever was written to the standard output stream of the child process.
- `.stderr: Buffer | string`
Contains whatever was written to the standard error stream of the child

process.

- `.status: number | null`

Contains the exit code of the child process or `null`. Either the exit code or the signal code are non-null.

- `.signal: string | null`

Contains the signal code of the child process or `null`. Either the exit code or the signal code are non-null.

- `.error?: Error`

This property is only created if spawning didn't work and then contains an Error object.

With the asynchronous `spawn()`, the child process ran concurrently and we could read standard I/O via streams. In contrast, the synchronous `spawnSync()` collects the contents of the streams and returns them to us synchronously (see next subsection).

3.1 When is the shell command executed?

When using the synchronous `spawnSync()`, the child process for the command is started synchronously. The following code demonstrates that:

```
import {spawnSync} from 'node:child_process';

spawnSync(
  'echo', ['Command starts'],
  {
    stdio: 'inherit',
    shell: true,
  }
);
console.log('After spawnSync()');
```

This is the output:

```
Command starts
After spawnSync()
```

3.2 Reading from stdout

The following code demonstrates how to read standard output:

```
import {spawnSync} from 'node:child_process';

const result = spawnSync(
  `echo rock && echo paper && echo scissors`,
  {
    stdio: ['ignore', 'pipe', 'inherit'], // (A)
    encoding: 'utf-8', // (B)
    shell: true,
  }
);
console.log(result);
assert.equal(
  result.stdout, // (C)
  'rock\npaper\nscissors\n'
);
assert.equal(result.stderr, null); // (D)
```

In line A, we use `options.stdio` to tell `spawnSync()` that we are only interested in standard output. We ignore standard input and pipe standard error to the parent process.

As a consequence, we only get a `result` property for standard output (line C) and the property for standard error is `null` (line D).

Since we can't access the streams that `spawnSync()` uses internally to handle the standard I/O of the child process, we tell it which encoding to use, via `options.encoding` (line B).

3.3 Sending data to the stdin of the child process

We can send data to the standard input stream of a child process via the `options` property `.input` (line A):

```
import {spawnSync} from 'node:child_process';

const result = spawnSync(
  `sort`,
  {
    stdio: ['pipe', 'pipe', 'inherit'],
    encoding: 'utf-8',
    input: 'Cherry\nApple\nBanana\n', // (A)
```

```
    }  
  );  
  assert.equal(  
    result.stdout,  
    'Apple\nBanana\nCherry\n'  
  );  
}
```

3.4 Handling unsuccessful exits (including errors)

There are three main kinds of unsuccessful exits (when the exit code isn't zero):

- The child process can't be spawned.
- An error happens in the shell.
- A process is killed.

3.4.1 The child process can't be spawned

If spawning fails, `spawn()` emits an `'error'` event. In contrast, `spawnSync()` sets `result.error` to an error object:

```
import {spawnSync} from 'node:child_process';  
  
const result = spawnSync(  
  'echo hello',  
  {  
    stdio: ['ignore', 'inherit', 'pipe'],  
    encoding: 'utf-8',  
    shell: '/bin/does-not-exist',  
  }  
);  
assert.equal(  
  result.error.toString(),  
  'Error: spawnSync /bin/does-not-exist ENOENT'  
);
```

3.4.2 An error happens in the shell

If an error happens in the shell, the exit code `result.status` is greater than zero and `result.signal` is null:

```
import {spawnSync} from 'node:child_process';

const result = spawnSync(
  'does-not-exist',
  {
    stdio: ['ignore', 'inherit', 'pipe'],
    encoding: 'utf-8',
    shell: true,
  }
);
assert.equal(result.status, 127);
assert.equal(result.signal, null);
assert.equal(
  result.stderr, '/bin/sh: does-not-exist: command not found\n'
);
```

3.4.3 A process is killed

If the child process is killed on Unix, `result.signal` contains the name of the signal and `result.status` is null:

```
import {spawnSync} from 'node:child_process';

const result = spawnSync(
  'kill $$',
  {
    stdio: ['ignore', 'inherit', 'pipe'],
    encoding: 'utf-8',
    shell: true,
  }
);

assert.equal(result.status, null);
assert.equal(result.signal, 'SIGTERM');
assert.equal(result.stderr, ''); // (A)
```

Note that no output was sent to the standard error stream (line A).

If we kill a child process on Windows:

- `result.status` is 1

- `result.signal` is `null`
- `result.stderr` is `''`

4 Asynchronous helper functions based on `spawn()`

In this section, we look at two asynchronous functions in module `node:child_process` that are based on `spawn()`:

- `exec()`
- `execFile()`

We ignore `fork()` in this blog post. Quoting [the Node.js documentation](#):

`fork()` spawns a new Node.js process and invokes a specified module with an IPC communication channel established that allows sending messages between parent and child.

4.1 `exec()`

```
exec(  
  command: string,  
  options?: Object,  
  callback?: (error, stdout, stderr) => void  
) : ChildProcess
```

`exec()` runs a command in a newly spawned shell. The main differences with `spawn()` are:

- In addition to returning a `ChildProcess`, `exec()` also delivers a result via a callback: Either an error object or the contents of `stdout` and `stderr`.
- Causes of errors: child process can't be spawned, shell error, child process killed.
 - In contrast, `spawn()` only emits `'error'` events if the child process can't be spawned. The other two failures are handled via exit codes and (on Unix) signal codes.
- There is no parameter `args`.
- The default for `options.shell` is `true`.

```
import {exec} from 'node:child_process';
```



```
const childProcess = exec(
  'echo Hello',
  (error, stdout, stderr) => {
    if (error) {
      console.error('error: ' + error.toString());
      return;
    }
    console.log('stdout: ' + stdout); // 'stdout: Hello\n'
    console.error('stderr: ' + stderr); // 'stderr: '
  }
);
```

`exec()` can be converted to a Promise-based function via `util.promisify()`:

- The `ChildProcess` becomes a property of the returned Promise.
- The Promise is settled as follows:
 - Fulfillment value: `{stdout, stderr}`
 - Rejection value: same value as parameter `error` of the callback but with two additional properties: `.stdout` and `.stderr`.

```
import * as util from 'node:util';
import * as child_process from 'node:child_process';

const execAsync = util.promisify(child_process.exec);

try {
  const resultPromise = execAsync('echo Hello');
  const {childProcess} = resultPromise;
  const obj = await resultPromise;
  console.log(obj); // { stdout: 'Hello\n', stderr: '' }
} catch (err) {
  console.error(err);
}
```

4.2 `execFile()`

`execFile(file, args?, options?, callback?): ChildProcess`

Works similarly to `exec()`, with the following differences:

- The parameter `args` is supported.

- The default for `options.shell` is `false`.

Like `exec()`, `execFile()` can be converted to a Promise-based function via `util.promisify()`.

5 Synchronous helper functions based on `spawnAsync()`

5.1 `execSync()`

```
execSync(  
  command: string,  
  options?: Object  
): Buffer | string
```

`execSync()` runs a command in a new child process and waits synchronously until that process exits. The main differences with `spawnSync()` are:

- Only returns the contents of `stdout`.
- Three kinds of failures are reported via exceptions: child process can't be spawned, shell error, child process killed.
 - In contrast, the result of `spawnSync()` only has an `.error` property if the child process can't be spawned. The other two failures are handled via exit codes and (on Unix) signal codes.
- There is no parameter `args`.
- The default for `options.shell` is `true`.

```
import {execSync} from 'node:child_process';  
  
try {  
  const stdout = execSync('echo Hello');  
  console.log('stdout: ' + stdout); // 'stdout: Hello\n'  
} catch (err) {  
  console.error('Error: ' + err.toString());  
}
```

5.2 `execFileSync()`

`execFileSync(file, args?, options?): Buffer | string`

Works similarly to `execSync()`, with the following differences:

- The parameter `args` is supported.
- The default for `options.shell` is `false`.

6 Useful libraries

6.1 `tinysh`: a helper for spawning shell commands

`tinysh` by Anton Medvedev is a small library that helps with spawning shell commands – e.g.:

```
import sh from 'tinysh';

console.log(sh.ls('-l'));
console.log(sh.cat('README.md'));
```

We can override the default options by using `.call()` to pass an object as `this`:

```
sh.tee.call({input: 'Hello, world!'}, 'file.txt');
```

We can use any property name and `tinysh` executes the shell command with that name. It achieves that feat via a [Proxy](#). This is a slightly modified version of the actual library:

```
import {execFileSync} from 'node:child_process';
const sh = new Proxy({}, {
  get: (_, bin) => function (...args) { // (A)
    return execFileSync(bin, args,
      {
        encoding: 'utf-8',
        shell: true,
        ...this // (B)
      }
    );
  },
});
```

In line A, we can see that if we get a property whose name is `bin` from `sh`, a function is returned that invokes `execFileSync()` and uses `bin` as the first argument.

Spreading `this` in line B enables us to specify options via `.call()`. The defaults come first, so that they can be overridden via `this`.

6.2 node-powershell: executing Windows PowerShell commands via Node.js

Using the library `node-powershell` on Windows, looks as follows:

```
import { PowerShell } from 'node-powershell';  
PowerShell.$`echo "hello from PowerShell"`;
```

7 How to choose between the functions of module `'node:child_process'`

General constraints:

- Should other asynchronous tasks run while the command is executed?
 - Use any asynchronous function.
- Do you only execute one command at a time (without async tasks in the background)?
 - Use any synchronous function.
- Do you want to access stdin or stdout of the child process via a stream?
 - Only asynchronous functions give you access to streams: `spawn()` is simpler in this case because it doesn't have a callback that delivers errors and standard I/O content.
- Do you want to capture stdout or stderr in a string?
 - Asynchronous options: `exec()` and `execFile()`
 - Synchronous options: `spawnSync()`, `execSync()`, `execFileSync()`

Asynchronous functions – choosing between `spawn()` and `exec()` or `execFile()`:

- `exec()` and `execFile()` have two benefits:
 - Failures are easier to handle because they are all reported in the same manner – via the first callback parameter.
 - Getting stdout and stderr as strings is easier - due to the callback.
- You can pick `spawn()` if those benefits don't matter to you. Its signature is simpler without the (optional) callback.

Synchronous functions – choosing between `spawnSync()` and `execSync()` or `execFileSync()`:

- `execSync()` and `execFileSync()` have two specialties:
 - They return a string with the content of stdout.

- Failures are easier to handle because they are all reported in the same manner – via exceptions.
- Pick `spawnSync()` if you need more information than `execSync()` and `execFileSync()` provide via their return values and exceptions.

Choosing between `exec()` and `execFile()` (the same arguments apply to choosing between `execSync()` and `execFileSync()`):

- The default for `options.shell` is `true` in `exec()` but `false` in `execFile()`.
- `execFile()` supports `args`, `exec()` doesn't.